BAB
VIII
SEGMENTASI CITRA (*IMAGE SEGMENTATION*)

Materi:

- Segmentasi Citra
- *Thresholding*
- Deteksi Tepi (*Edge Detection*)

Tujuan Pembelajaran:

- Mahasiswa dapat mengimplementasikan teknik *thresholding* dan *edge detection* menggunakan OpenCV-Python.

A. Penyajian

1. Segmentasi Citra

Segmentasi citra merupakan proses mempartisi citra menjadi beberapa daerah atau objek. Umumnya segmentasi bertujuan untuk memisahkan suatu objek citra dengan objek lainnya dan atau memisahkan objek citra dengan latar belakang citra. Segmentasi citra didasarkan pada dua hal yaitu diskontinuitas dan kemiripan intensitas piksel. Pendekatan diskontinuitas disebut juga pendekatan berbasis *edge* (*edge-based*), sementara pendekatan kemiripan intensitas, seperti *thresholding*, *clustering* (contoh: k-means), klasifikasi, pendekatan teori graf (contoh: GrabCut), *region growing*, dan *region splitting & merging*.

2. Thresholding

Thresholding merupakan teknik yang paling dasar untuk melakukan segmentasi citra. Teknik *thresholding* dapat digunakan untuk melakukan segmentasi citra berdasarkan warna tertentu atau berdasarkan suatu nilai piksel tertentu.

Dipindai dengan CamScanner

3. Deteksi Tepi (*Edge Detection*)

Deteksi tepi adalah operasi yang dijalankan untuk mendeteksi garis tepi yang membatasi dua wilayah citra homogen yang memiliki tingkat kecerahan yang berbeda. *Edge* adalah bagian dari citra di mana intensitas kecerahan berubah secara drastis. Tujuan dari deteksi tepi pada umumnya yaitu untuk mengurangi jumlah data pada citra pada langkah *image processing* berikutnya. Deteksi tepi adalah pendekatan yang sering digunakan untuk segmentasi citra digital dengan berdasarkan perubahan intensitas yang lokal. Ada beberapa pendekatan atau metode untuk deteksi tepi, yaitu *first order edge detection* dan *second order edge detection* [1].

a. First Order Derivative Edge Detection

Metode deteksi tepi yang termasuk ke dalam *first order derivative* adalah Roberts *operators*, Prewitt *operators*, Sobel *operators*, dan *First-order of gaussian* (FDOG). Pada modul ini akan dicoba untuk operator Sobel, Prewitt, dan Roberts. Setiap operator memiliki *row gradient* dan *column gradient*.

1) Roberts Operator

z_1	z_2	z_3
z_4	z_5	z_6
z_7	z_8	z_9

0	0	0
0	−1	0
0	0	1

$G_x = Z_9 - Z_5$

0	0	0
0	0	−1
0	1	0

$G_x = Z_8 - Z_6$

Gambar 52. Roberts Operator

Dipindai dengan CamScanner

2) Prewitt Operator

-1	-1	-1
0	0	0
1	1	1

$G_x = (Z_7 + Z_8 + Z_9)$
$-(Z_1 + Z_2 + Z_3) - G_x$

1	1	1
0	0	0
-1	-1	-1

$G_y = (Z_7 + Z_8 + Z_9)$
$-(Z_1 + Z_2 + Z_3) - G_x$

Gambar 53. Prewitt Operator

OpenCV-Python tidak menyediakan fungsi khusus untuk operator **roberts** dan **prewitt**. Namun dapat menggunakan fungsi **filter2D** yang terdapat pada OpenCV-Python dengan mendefinisikan sendiri kernel-nya, sesuai dengan operator yang ingin digunakan. Fungsi **filter2D** telah dijelaskan pada Modul VI: *Image Enhancement* dengan *Mask Processing*:

```
dst = cv.filter2D(src, ddepth, kernel, anchor, delta,
borderType)
```

3) Sobel Operator

Operator Sobel adalah operasi smoothing Gaussian bersama diferensiasi, sehingga lebih tahan terhadap noise. Arah turunan dapat kita tentukan, baik secara vertikal maupun horizontal (masing-masing dengan argumen x order dan y order). Dokumen lebih detail terkait kernel yang digunakan dapat dilihat pada:

- https://docs.opencv.org/4.1.2/d4/d86/group__imgproc__filter.html#gacea54f142e81b6758cb6f375ce782c8d.

Berikut contoh dengan parameter sobel (xorder = 1, yorder = 0, ksize = 3) atau (xorder = 0, yorder = 1, ksize = 3).

Dipindai dengan CamScanner

-1	-2	-1
0	0	0
1	2	1

$G_x = (Z_7 + 2Z_8 + Z_9)$
$-(Z_1 + 2Z_2 + Z_3) - G_x$

1	2	1
0	0	0
-1	-2	-1

$G_y = (Z_7 + 2Z_8 + Z_9)$
$-(Z_1 + 2Z_2 + Z_3) - G_x$

Gambar 54. Sobel Operator

Pada OpenCV-Python dapat menggunakan fungsi sebagai berikut [2]:

```
dst = cv.Sobel(src, ddepth, dx, dy, ksize)
```

keterangan

dst : citra *output* dengan ukuran yang sama dan jumlah *channel* yang sama dengan src.
src : citra input
dx : order dari *derivative* x.
dy : order dari *derivative* y.
ksize : ukuran kernel Sobel (1, 3, 5, atau 7).

b. Second Order Derivative Edge Detection

1) Laplacian Derivatives

Laplacian gambar dappat dihitung dengan,

$$\Delta src = \frac{\partial^2 src}{\partial x^2} + \frac{\partial^2 src}{\partial y^2}$$

di mana setiap turunan ditemukan menggunakan turunan Sobel. Jika ksize = 1, maka kernel berikut digunakan untuk memfilter:

$$kernel = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Dipindai dengan CamScanner

Pada OpenCV-Python dapat menggunakan fungsi sebagai berikut:

```
dst = cv.Laplacian(src, ddepth, ksize)
```

keterangan

dst : citra *output* dengan ukuran yang sama dan jumlah *channel* yang sama dengan src.

src : citra input.

ksize : Ukuran aperture yang digunakan untuk menghitung filter *derivative* kedua. Ukuran harus bernilai positif dan ganjil, untuk nilai default adalah 1.

2) Canny detection

Materi lebih lanjut tentang Canny dapat dilihat pada file Canny Operator di LMS. Untuk melakukan canny, dilakukan 5 langkah berikut [2]:

a) *Smoothing*, melakukan *blurring* untuk menghilangkan *noise*. Karena deteksi tepi rentan terhadap noise pada gambar, langkah pertama adalah menghilangkan noise pada gambar dengan filter Gaussian 5x5.

b) Menemukan gradien

Gambar yang dihaluskan kemudian dilakukan filter dengan kernel Sobel pada order horizontal (G_x) dan vertikal (G_y) untuk mendapatkan turunan pertama. Dari dua gambar ini, kita dapat menemukan gradien tepi dan arah untuk setiap piksel sebagai berikut:

$$Edge_Gradient\ (G) = \sqrt{G_x^2 + G_y^2}$$

$$Angle\ (\theta) = \tan^{-1}\left(\frac{G_y}{G_x}\right)$$

Dipindai dengan CamScanner

threshold1 : *threshold* pertama untuk prosedur hysteresis.
threshold2 : *threshold* kedua untuk prosedur hysteresis.

B. Latihan

1. Baca gambar 'monas.jpg', lalu terapkan deteksi *edge* menggunakan operator sobel pada order horizontal (G_x), vertical (G_y), dan magnitude-nya, serta lakukan deteksi *edge* dengan menggunakan operator *Laplacian Derivatives*. Lalu tampilkan kelima gambar (gambar asli, gambar hasil filter sobel derivative x, gambar hasil filter sobel derivative y, gambar hasil magnitude, serta hasil filter Laplacian Derivatives) dengan menggunakan cv.imshow dan subplot matplotlib.

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread('monas.jpg',0)

laplacian64f = cv.Laplacian(img,cv.CV_64F)
abs_laplacian64f = np.absolute(laplacian64f)
laplacian_8u = np.uint8(abs_laplacian64f)

sobelx64f = cv.Sobel(img,cv.CV_64F,1,0,ksize=3)
abs_sobelx64f = np.absolute(sobelx64f)
sobelx_8u = np.uint8(abs_sobelx64f)

sobely64f = cv.Sobel(img,cv.CV_64F,0,1,ksize=3)
abs_sobely64f = np.absolute(sobely64f)
sobely_8u = np.uint8(abs_sobely64f)

magnitudesobel = cv.magnitude(sobelx64f,sobely64f)
abs_sobel64f = np.absolute(magnitudesobel)
sobel_8u = np.uint8(abs_sobel64f)
```


Dipindai dengan CamScanner

```
plt.subplot(3,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,2),plt.imshow(laplacian_8u,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,3),plt.imshow(sobelx_8u,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,4),plt.imshow(sobely_8u,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,5),plt.imshow(sobel_8u,cmap = 'gray')
plt.title('Sobel Magnitude'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Laplacian', laplacian_8u)
cv.imshow('Sobel X', sobelx_8u)
cv.imshow('Sobel Y', sobely_8u)
cv.imshow('Sobel Magnitude', sobel_8u)
cv.waitKey(0)
cv.destroyAllWindows()
```

2. Baca gambar 'monas.jpg', lalu terapkan deteksi edge menggunakan *canny detection*.

```
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('monas.jpg',0)
edges = cv.Canny(img,100,200)
plt.subplot(121),plt.imshow(img,cmap = 'gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(edges,cmap = 'gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Canny', edges)

cv.waitKey(0)
cv.destroyAllWindows()
```


Dipindai dengan CamScanner

```
plt.subplot(3,2,1),plt.imshow(img,cmap = 'gray')
plt.title('Original'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,2),plt.imshow(laplacian_8u,cmap = 'gray')
plt.title('Laplacian'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,3),plt.imshow(sobelx_8u,cmap = 'gray')
plt.title('Sobel X'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,4),plt.imshow(sobely_8u,cmap = 'gray')
plt.title('Sobel Y'), plt.xticks([]), plt.yticks([])
plt.subplot(3,2,5),plt.imshow(sobel_8u,cmap = 'gray')
plt.title('Sobel Magnitude'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Laplacian', laplacian_8u)
cv.imshow('Sobel X', sobelx_8u)
cv.imshow('Sobel Y', sobely_8u)
cv.imshow('Sobel Magnitude', sobel_8u)
cv.waitKey(0)
cv.destroyAllWindows()
```

2. Baca gambar 'monas.jpg', lalu terapkan deteksi edge menggunakan *canny detection*.

```
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('monas.jpg',0)
edges = cv.Canny(img,100,200)
plt.subplot(121),plt.imshow(img,cmap = 'gray')
plt.title('Original Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122),plt.imshow(edges,cmap = 'gray')
plt.title('Edge Image'), plt.xticks([]), plt.yticks([])
plt.show()

cv.imshow('Original', img)
cv.imshow('Canny', edges)

cv.waitKey(0)
cv.destroyAllWindows()
```

Dipindai dengan CamScanner

C. Lembar Kerja Praktikum

1. Indonesia terkenal dengan keanekaragaman hayatinya, baik hewan dan tumbuhan. Banyak jenis tumbuhan khas Indonesia yang mungkin ada di sekitar Kita. Tugas Anda adalah ambil foto di sekitar terkait tumbuhan khas Indonesia. Foto yang diambil adalah bagian daun tumbuhan dengan latar belakang warna putih (daun dapat dipetik terlebih dahulu, lalu letakan di atas kertas putih). Daftar tumbuhan dapat dilihat pada halaman website ini (https://rimbakita.com/daftar-nama-tumbuhan-di-indonesia/).
   a. Lakukan deteksi tepi dengan menggunakan operator sobel pada *order horizontal* (G_x), *vertical* (G_y), dan *magnitude*-nya, lakukan juga deteksi *edge* dengan menggunakan operator *laplacian derivatives* dan *canny detection.*
   b. Lakukan segmentasi terhadap foto tersebut sedemikian sehingga latar foto bernilai 0 dan objek daun sesuai dengan intensitas objek tersebut. Pemrosesan citra menggunakan *grayscale*.

Referensi

[1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.

[2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

Dipindai dengan CamScanner

BAB
IX
MORFOLOGI

Materi:

- Definisi dan Fungsi Morfologi
- Dilasi
- Erosi
- *Opening*
- *Closing*

Tujuan Pembelajaran:

- Mahasiswa dapat mengimplementasikan operasi-operasi morfologi di OpenCV-Python.

A. Penyajian

1. Definisi dan Fungsi Morfologi

Kata morfologi biasa kita dengar di cabang biologi yaitu yang berhubungan dengan bentuk dan struktur hewan dan tumbuhan. Sama halnya dalam konteks morfologi matematis yaitu sebagai alat untuk mengekstraksi komponen gambar yang berguna dalam representasi dan deskripsi bentuk wilayah, seperti batas dan kerangka. Teknik morfologi juga berguna untuk *pre-processing* dan *post-processing* citra, seperti penipisan dan pemangkasan, misalnya untuk untuk memperbaiki hasil segmentasi [1]. Morfologi adalah proses mengidentifikasi bentuk dengan basis region (citra bertipe biner & *grayscale*). Adapun operasi-operasi morfologi yang sering digunakan ialah dilasi, erosi, *opening*, dan *closing*.

Dipindai dengan CamScanner

2. Dilasi

Merupakan proses penggabungan titik-latar 0 menjadi bagian dari objek 1 berdasarkan *structuring element* (SE) yang digunakan. Notasi dilasi dapat ditulis berdasarkan persamaan di bawah ini.

$$D(A,S) = A \oplus S$$

Keterangan:

D(A,S) : Fungsi dilasi
A : Citra
S : *Structure element*
$\oplus$: Simbol dilasi

Fungsi Dilasi pada OpenCV [2]:

```
dst = cv.dilate(src, kernel, anchor, iterations, borderType, borderValue)
```

Keterangan:

src	:	input image; jumlah channel bisa berapa saja, **tetapi** kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.
dst	:	output image dengan ukuran dan tipe yang **sama** dengan src.
kernel	:	*structuring element* yang digunakan untuk dilasi; **jika** element = Mat (), *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat **menggunakan** getStructureElement. (https://docs.opencv.org/4.1.2/d4/d86/group__imgproc__filter.html)
anchor	:	posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.
iterations	:	*number of times dilation is applied.*
borderType	:	metode ekstrapolasi piksel, lihat BorderTypes (https://docs.opencv.org/4.1.2/d2/de8/group__core__array.html)
borderValue	:	*border value* untuk nilai border yang konstan

Dipindai dengan CamScanner

3. Erosi

Merupakan proses penghapusan titik-titik objek (1) menjadi bagian dari latar (0), berdasarkan *structuring element* (SE) yang digunakan. Notasi erosi dapat ditulis berdasarkan persamaan di bawah ini.

$$E(A,S) = A \otimes S$$

Keterangan:

E(A,S) : Fungsi erosi
A : Citra
S : *Structure element*
⊗ : Simbol erosi

Fungsi Erosi pada OpenCV [2]:

```
Dst = cv.erode(src, kernel, anchor, iterations, borderType, borderValue)
```

Keterangan:

src	:	input image; jumlah channel bisa berapa saja, tetapi kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.
dst	:	output image dengan ukuran dan tipe yang sama dengan src.
kernel	:	*structuring element* yang digunakan untuk dilasi; jika element = Mat (), *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat menggunakan getStructureElement. (https://docs.opencv.org/4.1.2/d4/d86/group__imgproc__filter.html)
anchor	:	posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.
iterations	:	number of times dilation is applied.
borderType	:	metode ekstrapolasi piksel, lihat BorderTypes (https://docs.opencv.org/4.1.2/d2/de8/group__core__array.html)
borderValue	:	*border value* untuk nilai border yang konstan

Dipindai dengan CamScanner

4. Opening dan Closing

Opening adalah proses erosi yang diikuti dengan dilasi. Efek yang dihasilkan adalah menghilangnya objek-objek kecil dan kurus, memecah objek pada titik-titik yang kurus, dan secara umum menghaluskan batas objek besar tanpa mengubah area objek secara signifikan. *Opening* berguna untuk **menghaluskan citra, menghilangkan tonjolan yang tipis,** dan **memisahkan objek**. Notasi *opening* dapat ditulis berdasarkan persamaan di bawah ini.

$$A \circ S = (A \otimes S) \oplus S$$

Closing adalah proses dilasi yang diikuti dengan erosi. Efek yang dihasilkan adalah mengisi lubang kecil pada objek, menggabungkan objek-objek yang berdekatan, dan secara umum menghaluskan batas dari objek besar tanpa mengubah area objek secara signifikan. Teknik ini berguna untuk menghaluskan citra, menghilangkan lubang yang kecil. Notasi *closing* dapat ditulis berdasarkan persamaan di bawah ini.

$$A \circ S = (A \oplus S) \otimes S$$

Fungsi Opening pada OpenCV [2]:

```
dst = cv.morphologyEx(src, op, kernel, anchor, iterations, borderType,
borderValue)
```

eterangan:

: input image; jumlah channel bisa berapa saja, tetapi kedalamannya harus salah satu dari CV_8U, CV_16U, CV_16S, CV_32F or CV_64F.

: output image dengan ukuran dan tipe yang sama dengan src.

: tipe dari operasi morphological, untuk closing: cv.MORPH_CLOSE dan untuk opening: cv.MORPH_OPEN, lihat **MorphTypes** (https://docs.opencv.org/4.1.2/d4/d86/group__imgproc__filter.html)

Dipindai dengan CamScanner

kernel	:	*structuring element* yang digunakan untuk dilasi; jika element = Mat (), *structuring element* persegi 3 x 3 digunakan. Kernel dapat dibuat menggunakan getStructureElement. (https://docs.opencv.org/4.1.2/d4/d86/group__imgproc__filter.html)
anchor	:	posisi jangkar di dalam elemen; nilai default (-1,-1) berarti bahwa jangkar berada di pusat elemen.
iterations	:	number of times dilation is applied.
borderType	:	metode ekstrapolasi piksel, lihat BorderTypes (https://docs.opencv.org/4.1.2/d2/de8/group__core__array.html)
borderValue	:	*border value* untuk nilai border yang konstan

Contoh Structuring Element

```
# Rectangular Kernel
>>> cv.getStructuringElement(cv.MORPH_RECT,(5,5))
array([[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1]], dtype=uint8)
# Elliptical Kernel
>>> cv.getStructuringElement(cv.MORPH_ELLIPSE,(5,5))
array([[0, 0, 1, 0, 0],
[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1],
[1, 1, 1, 1, 1],
[0, 0, 1, 0, 0]], dtype=uint8)
# Cross-shaped Kernel
>>> cv.getStructuringElement(cv.MORPH_CROSS,(5,5))
array([[0, 0, 1, 0, 0],
[0, 0, 1, 0, 0],
[1, 1, 1, 1, 1],
[0, 0, 1, 0, 0],
[0, 0, 1, 0, 0]], dtype=uint8)
```


Dipindai dengan CamScanner

B. Latihan

1. Berikut ini merupakan latihan membuat program yang dapat melakukan dilasi, erosi, *opening*, *closing*.

Morfologi.py

```
import numpy as np
import cv2 as cv

def morfologi(img, se):
dst_erosi = cv.erode(img, se, iterations = 1)
dst_dilate = cv.dilate(img, se, iterations = 1)
dst_opening = cv.morphologyEx(img, cv.MORPH_OPEN, se, iterations =
1)
dst_closing = cv.morphologyEx(img, cv.MORPH_CLOSE, se, iterations =
1)
return dst_erosi, dst_dilate, dst_opening, dst_closing

def main():
img1 = cv.imread('circbw.tif',0)
s_element = cv.getStructuringElement(cv.MORPH_RECT,(3,3))
dst_erosi, dst_dilate, dst_opening, dst_closing = morfologi(img1,
s_element)

cv.imshow('dst_erosi',dst_erosi)
cv.imshow('dst_dilate',dst_dilate)
cv.imshow('dst_opening',dst_opening)
cv.imshow('dst_closing',dst_closing)
cv.waitKey(0)
cv.destroyAllWindows()
```

Dipindai dengan CamScanner

2. Ingat kembali pada latihan di Modul 3: Ruang Warna Pengolahan Citra, dalam latihan tersebut kita mencoba mensegmentasi objek apel warna merah. Berikut ini adalah kode programnya:

```
import numpy as np
import cv2 as cv

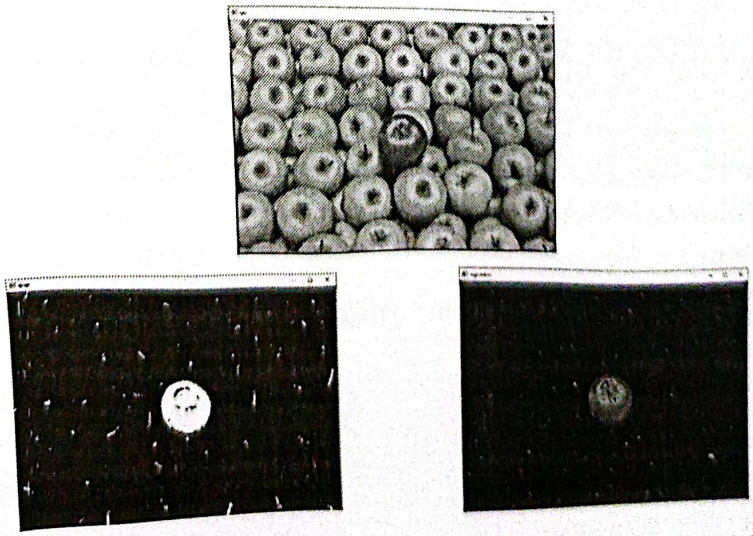
#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
ycrcb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])
# Threshold the HSV image to get only blue colors
mask = cv.inRange(ycrcb, lower_blue, upper_blue)

res = cv.bitwise_and(rgb,rgb, mask= mask)

cv.imshow('src', rgb)
cv.imshow('rgb', mask)
cv.imshow('objek', res)
cv.waitKey(0)
cv.destroyAllWindows()
```

Output:



Gambar 57. Citra Output Hasil Segmentasi tanpa Dilakukan Morfologi.

Dipindai dengan CamScanner

Dari hasil menggunakan segmentasi masih terlihat belum sempurna, yaitu masih ada noise pada latar belakang dan bentuk apel yang masih ada lubang kosong di dalamnya. Hasil segmentasi tersebut dapat diperbaiki dengan morfologi sebagai berikut.

```
import numpy as np
import cv2 as cv

#membaca citra RGB
rgb = cv.imread('red_apple.jpg')
ycrcb = cv.cvtColor(rgb, cv.COLOR_BGR2YCrCb)

lower_blue = np.array([40,150,50])
upper_blue = np.array([200,255,150])
# Threshold the HSV image to get only blue colors
img_biner = cv.inRange(ycrcb, lower_blue, upper_blue)

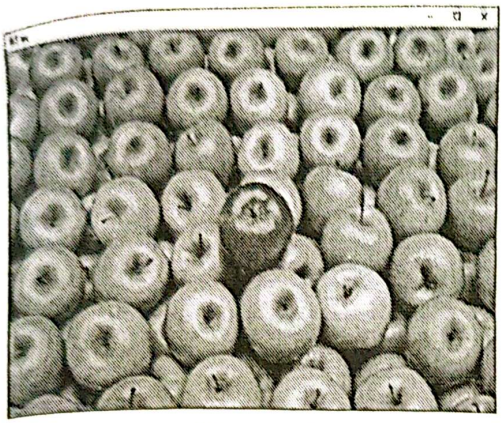
#mendefinisikan structuring element 3x3 dan 5x5
se_3 = cv.getStructuringElement(cv.MORPH_RECT,(3,3))
se_5 = cv.getStructuringElement(cv.MORPH_RECT,(5,5))
#opearsi morfologi
dst_dilate = cv.dilate(img_biner, se_3, iterations = 1)
dst_erosi = cv.erode(dst_dilate, se_3, iterations = 2)
dst_dilate2 = cv.dilate(dst_erosi, se_5, iterations = 2)
dst_erosi2 = cv.erode(dst_dilate2, se_5, iterations = 3)
dst_dilate3 = cv.dilate(dst_erosi2, se_3, iterations = 1)

res = cv.bitwise_and(rgb,rgb, mask= dst_dilate3)

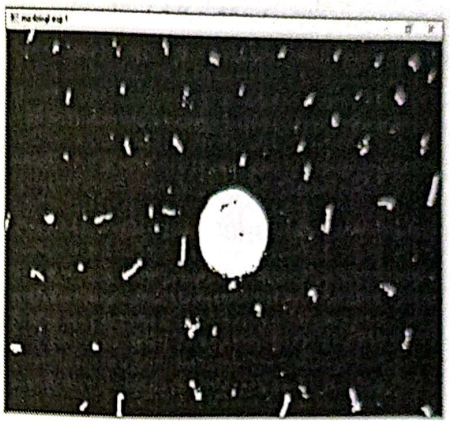
cv.imshow('src', rgb)
cv.imshow('rgb', dst_dilate3)
cv.imshow('objek', res)
cv.waitKey(0)
cv.destroyAllWindows()
```


Dipindai dengan CamScanner

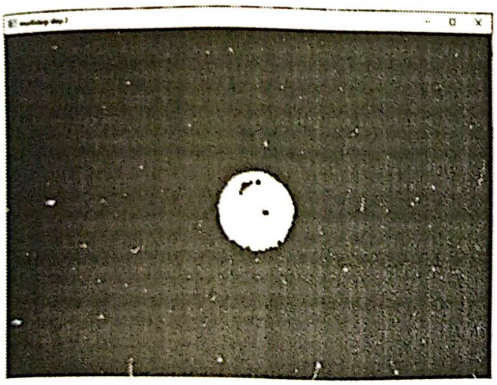
Keluaran:



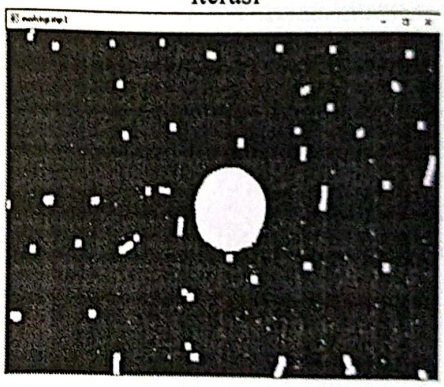
Citra awal



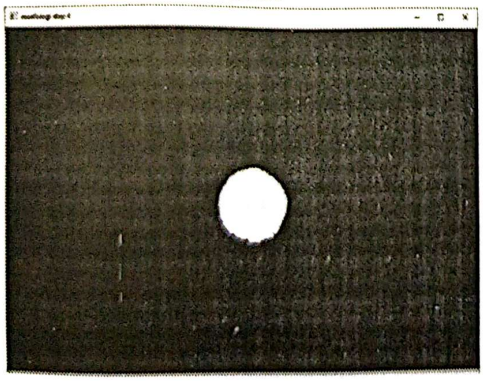
Hasil Dilasi dengan SE 3x3 satu kali iterasi



Hasil Erosi dengan SE 3x3 dua kali iterasi



Hasil Dilasi dengan SE 5x5 dua kali iterasi



Hasil Erosi dengan SE 5x5 tiga kali iterasi



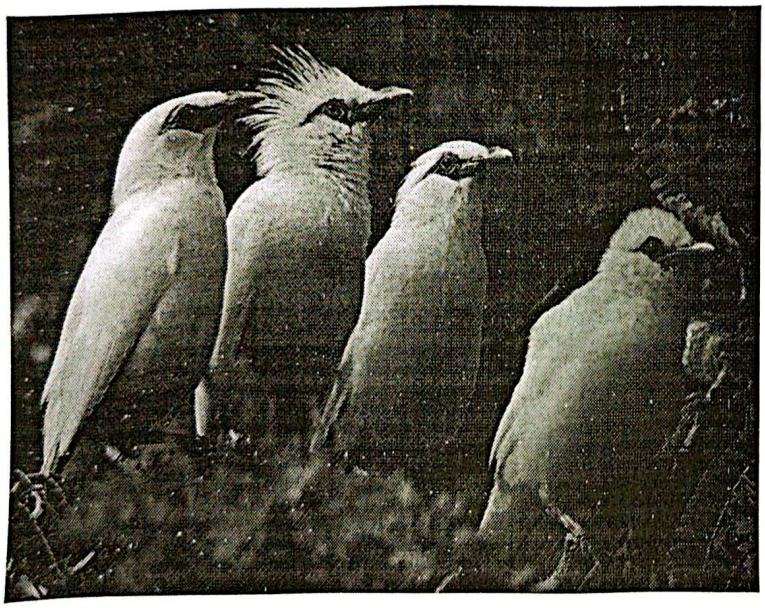
Citra Hasil

Gambar 58. Citra Output Latihan No.2.

Dipindai dengan CamScanner

C. Lembar Kerja Praktikum

1. Burung jalak bali merupakan satu-satunya satwa endemik Pulau Bali yang masih tersisa. Sebagai mahasiswa informatika yang memiliki jiwa bela Negara, ingin berkontribusi terhadap kelestarian burung tersebut di Indonesia. Maka akan dibuat aplikasi berbasis pengolahan citra digital untuk mendeteksi jumlah burung jalak bali yang ada di suatu wilayah. Program membaca gambar, dan keluarannya adalah jumlah burung jalak bali yang ada di foto tersebut. Gunakan Teknik image enhancement, image restoration, segmentasi citra, post-processing dengan morfologi, dan hitung jumlah burung yang ada di gambar tersebut.



Gambar 59. Foto Burung Jalak Bali

Referensi

[1] R. C. Gonzalez and R. E. Woods, "Digital Image Processing, 4th Edition," *J. Electron. Imaging*, p. 1019, Jan. 2018.

[2] A. Mordvintsev and A. K, "OpenCV-Python Tutorials," 2019. https://docs.opencv.org/4.1.2/d6/d00/tutorial_py_root.html.

Dipindai dengan CamScanner